Anomaly Detection: Spotting the Unusual in Data

From Intuition and Time Series to a Streaming Detector

Stephen Hailes

19 October 2026

Department of Computer Science, UCL

Session Plan

Introduction: What and Why

Foundations of Detection

Judging a Detector

Bioreactor Simulator Context

Getting Live Data via MQTT

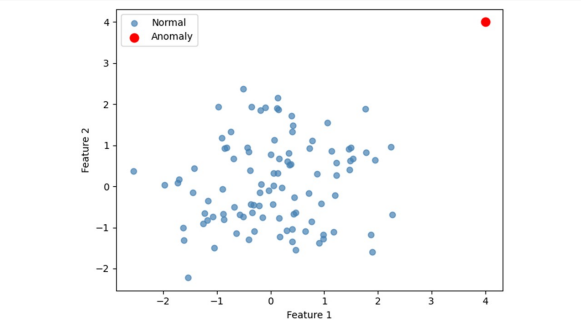
Wrap-Up

Introduction: What and Why

What is Anomaly Detection?

Simple definition

Anomaly detection is the identification of data points or patterns that deviate significantly from expected or typical behaviour.



Anomalies and Faults

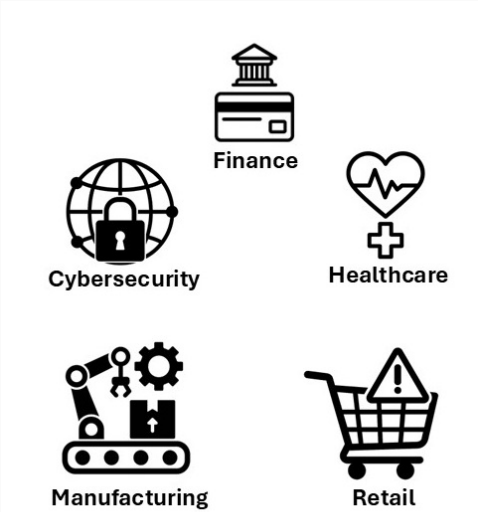
- A **fault** is something wrong with the system: a biased sensor, a weak heater, a sticking valve.
- An **anomaly** is data that looks unusual compared with normal behaviour.

- ***They are related, but not the same***:
  - a fault can produce **no visible anomaly**
    e.g. if a controller compensates for it;
  - an anomaly can occur with **no fault present**
    e.g. after a fault has cleared, or when a setpoint changes.

- A detector can only see anomalies. We want it to tell us about faults.

Where is Anomaly Detection Used?

- **Finance:** detecting fraudulent activity
- **Healthcare:** rare diseases or abnormal readings
- **Cybersecurity:** intrusion and malware detection
- **Manufacturing:** predictive fault monitoring
- **Process control:** chemical plants, water treatment, bioreactors
- **Retail:** unusual purchasing behaviour



Why Does It Matter?

- **Safety:** catch abnormal behaviour before accidents occur.
- **Reliability:** maintain stable operation and product quality.
- **Maintenance:** preventative rather than reactive.
- **Cost:** reduce downtime and material waste.
- **Security:** detect cyber-physical manipulation.

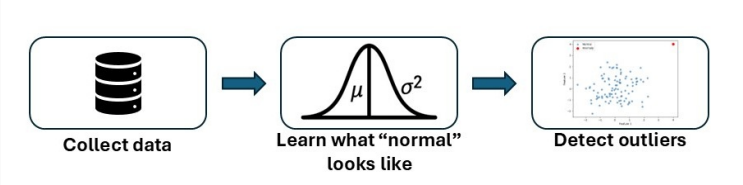
The Economics of Anomalies

- The world's 500 largest firms lose **$1.4 trillion / year** to unplanned downtime (≈ 11% of revenue). [Siemens 2024]
- A large plant loses on average **$253 million / year**; in automotive, downtime costs **$2.3 million per hour**. [Siemens 2024]
- Cost per hour of downtime: a **median of $125 k** across 11 industrial sectors [ABB 2023], but a **mean of $1.7 million** in US, UK and German manufacturing [Fluke 2025].
- Anomaly detection market ≈ **$7.5 B in 2026**, growing at about **15 to 17% a year**, projected to reach **$14.6 B by 2030** to **$17.7 B by 2032**. [Research and Markets 2026; GVR]

Types of Anomalies

- **Point anomaly:** single measurement deviating strongly from baseline.
- **Contextual anomaly:** value abnormal only within its context (e.g. time of day).
- **Collective anomaly:** a subsequence of otherwise normal points forming an abnormal pattern.

How Does It Work?



1. **Collect** representative data from sensors or logs (or MQTT in our case).
2. **Learn** what "normal" looks like, statistically or via models.
3. **Detect** deviations: score new data against what is normal, and decide when to raise an alarm.

Popular Methods (Simplified)

- **Statistical:** measure how far a value is from its usual range, in units of its typical spread (z-scores, coming up shortly).
- **Prediction-based:** predict the next value from recent values, and flag large prediction errors. (Names you may meet: moving averages, ARIMA, Kalman filters.)
- **Machine learning:** learn the overall shape of normal data, then flag points that do not fit it.
- **Deep learning:** neural networks trained to reproduce or predict normal data; large errors mean something unusual.

Challenges in Anomaly Detection

- **Defining normal:** what is normal depends on context, and changes when the setpoint changes.
- **Defining anomalous:** anomalous data may only appear some time after a fault starts, or even after it has ended.
- **Time:** each reading depends on the ones before it, so readings cannot be judged one at a time.
- **Rare faults:** few examples of anomalies, so the data are unbalanced.
- **Trade-off:** false alarms versus missed detections.
- **Drift:** systems change over time (concept drift).

Foundations of Detection

Time Series Basics

A time series is a sequence $\{x_t\}_{t=1}^{T}$ sampled in time. In the exercise, t counts the one-second summary messages. Three important properties are:

- **Level and spread:** for example the mean $\mu = \mathbb{E}[x]$ and variance $\sigma^2 = \mathrm{Var}(x)$ or, robustly, the median and median absolute deviation (MAD). See later.
- **Dependence:** how closely each value can be predicted from one k steps earlier, measured by the autocorrelation $\rho(k) = \mathrm{corr}(x_t, x_{t-k})$.
- **Stationarity:** the mean and variance stay stable over time.

What Should We Score? Residuals

- Predict what value would be given at this time through normal behaviour, $\hat{x}_t$.
- Score the **residual**:

$$r_t = x_t - \hat{x}_t$$

- Different methods differ only in how they predict:
  - $\hat{x}_t = \mu$, the usual level: is the value far from the average?
  - $\hat{x}_t = x_{t-1}$, the last value: did it change unusually fast?
  - $\hat{x}_t = f(y_t)$, from another signal y_t: does it agree with a signal it is normally related to?
  - $\hat{x}_t$ from a weighted sum of recent values and recent inputs: does it follow its usual pattern over time? (More on this shortly.)
- If the prediction is good, the residuals of normal data are close to **independent noise with zero mean**, even when x_t itself is neither stationary nor independent.

Standardising the Residual

z-score of feature i

$$z_{t,i} = \frac{r_{t,i}}{\sigma_i} = \frac{x_{t,i} - \hat{x}_{t,i}}{\sigma_i}$$

Pooled score across d features

$$s_t = \sqrt{\sum_{i=1}^{d} z_{t,i}^2}$$

A *feature* is each quantity we score, such as the residual of one signal. σ_i is the spread of its residuals on fault-free data, or a robust estimate of it (next slide).

Question

Suppose you compute a plain z-score of the measured temperature, $\hat{x}_t = \mu$.

The thermistor starts to read 5 °C low. What does your z-score do over the next few minutes? Think about the controller.

Robust Estimates and Redundancy

- Training data contain outliers, so estimate both the centre and the spread robustly. Over the fault-free training residuals of feature i:

$$m_i = \mathrm{median}\{r_{t,i}\} \qquad \text{in place of the mean}$$

$$\mathrm{MAD}_i = \mathrm{median}\{|r_{t,i} - m_i|\}$$

$$\sigma_i \approx 1.4826 \times \mathrm{MAD}_i \qquad \text{in place of the standard deviation}$$

$$z_{t,i} = \frac{r_{t,i} - m_i}{1.4826 \times \mathrm{MAD}_i}$$

- Learn these once and keep them fixed: a baseline that keeps adapting will learn the fault as normal.
- Avoid redundant or correlated features: they count the same evidence twice.
- Handle missing values gracefully.

What Is Likely to Work?

- The z-score and modified z-score will not work well if used naively. They are only as good as the prediction $\hat{x}_t$ behind them, and under feedback control a constant mean or median is rarely a good prediction.
- You might be tempted to start with a z-score of the raw readings because it is simple. That is fine as a first step for seeing the data, but expect it to catch only the moments when a fault starts or clears.
- You need to think about residuals the controller does not naturally hide.

Better Predictions: Models with Inputs

- Under feedback control, what the controller is doing tells us what the system is being asked to do. Use it as an **input** to the prediction.
- A model with inputs predicts from recent values x and recent inputs u:

$$\hat{x}_t = c + \sum_{i=1}^{p} a_i \, x_{t-i} + \sum_{j=1}^{q} b_j \, u_{t-j}$$

  with the coefficients fitted to fault-free data by least squares.
- For example, predict the temperature from past temperatures and the heater duty actually applied.
- If the sensor or the heater misbehaves, the measurements stop agreeing with the model, and the residual stays away from zero for as long as the fault lasts.

What Is a Decision Rule?

- A **decision rule** turns a continuous score into a yes or no decision.
- The simplest compares the score with a **threshold**:

  At each time t : if $s_t \geq \tau$ then label as anomaly; otherwise normal.

- This is a **statistical test**: τ sets the false alarm rate, the probability that normal data scores above it,

  $$\alpha = P(s_t \geq \tau \mid \text{normal}).$$

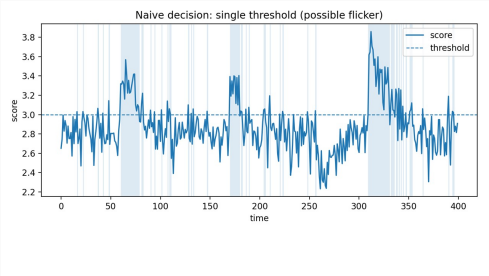
- To choose τ, score the fault-free training data and set τ above, say, 99.9% of those scores.
- Higher τ gives fewer false alarms but more missed anomalies, and lower τ the reverse.

Naive Thresholding

- A simple rule:

  If $s_t \geq \tau$ then alarm, else normal.

- But real data are noisy: scores often fluctuate around the threshold τ.
- Problem: when s_t hovers near τ, the alarm can flicker on and off.
- This instability makes alarms unreliable and distracting in practice.



Single threshold: the alarm flickers as the score crosses τ repeatedly.

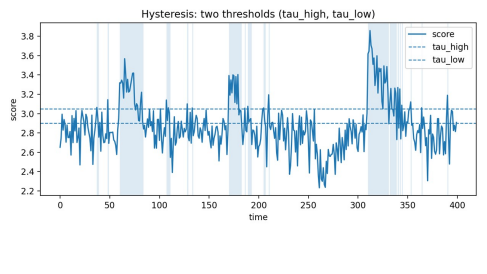
Hysteresis: Two Thresholds

- Adds stability by defining two thresholds:

$$\text{Raise alarm if } s_t \geq \tau_{\text{high}},$$

$$\text{Clear alarm if } s_t \leq \tau_{\text{low}}.$$

- The gap $\tau_{\text{high}} - \tau_{\text{low}}$ is the hysteresis band.
- Prevents frequent switching when the score fluctuates near the boundary.



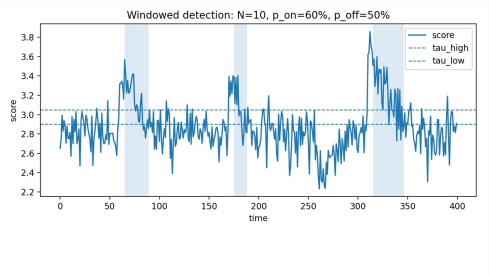
Two-threshold hysteresis: the alarm changes state only outside the band.

Windowed Detection

- Decide on the last N samples, not on one.
- Raise the alarm if at least k of the last N scores exceed τ; clear it when most fall below τ_{low}.
- If each normal sample exceeds τ independently with probability α:

$$P(\text{false alarm}) = \sum_{j=k}^{N} \binom{N}{j} \alpha^j (1-\alpha)^{N-j}$$

- e.g. $\alpha = 0.01$, $k = 3$, $N = 10$: about 10^{-4}.



Windowed detection: the alarm depends on the proportion of high scores within a time window.

Judging a Detector

Assessing Detector Performance

- We need objective measures of how well a detector distinguishes normal from faulty behaviour.
- Good detectors should:
  - **raise alarms** when faults occur;
  - **stay quiet** when the system behaves normally.
- Metrics must capture both: how often alarms are right (**precision**), and how many faults are caught (**recall**).
- Detectors should also raise alarms *quickly*, but that is for another day.

Question
Your detector sees anomalies, but you will score it against fault labels. When exactly should it count as having "caught" a fault?

Precision: How Reliable Are Our Alarms?

Precision: "When I raise an alarm, how often am I right?"

- High precision means few false alarms.
- It measures the **reliability** or **purity** of detected events.
- Example: 10 alarms raised, 8 of them genuine, so precision $= 0.8$.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- TP: true positives (correct alarms)
- FP: false positives (false alarms)

Recall: How Complete Is Our Detection?

Recall: "Of all real faults, how many did I detect?"

- High recall means few missed faults.
- It measures the **completeness** or **sensitivity** of the detector.
- Example: 12 faults occurred, 8 detected, so recall ≈ 0.67.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- TP: true positives (correct alarms)
- FN: false negatives (missed faults)

F_1 Score: Balancing Precision and Recall

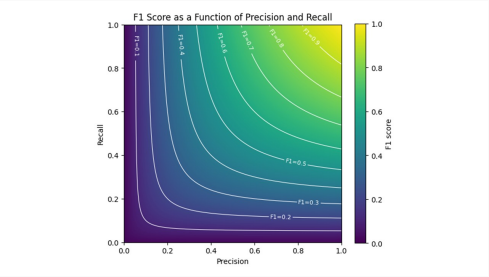
F_1 combines precision and recall into a single performance measure

- High only when both are high. If either precision or recall is low, F_1 falls sharply.
- $$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$
- Example values:

$$P = 0.8,\ R = 0.8 \Rightarrow F_1 = 0.8$$

$$P = 0.9,\ R = 0.3 \Rightarrow F_1 = 0.45$$

$$P = 0.4,\ R = 0.9 \Rightarrow F_1 \approx 0.55$$



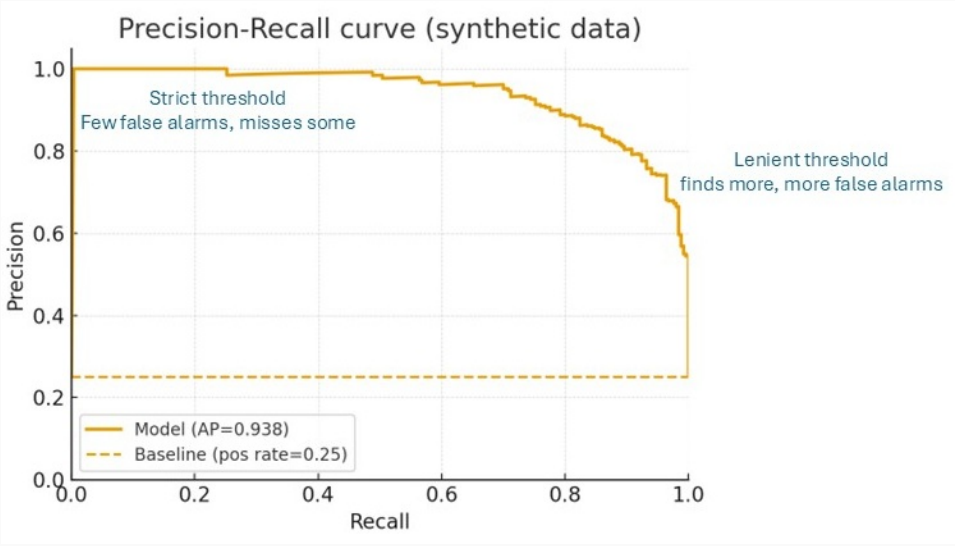
F_1 score surface: colour indicates F_1 for combinations of precision and recall.

Creating a Precision-Recall Curve

A precision-recall curve shows how precision and recall change as we vary the decision threshold.

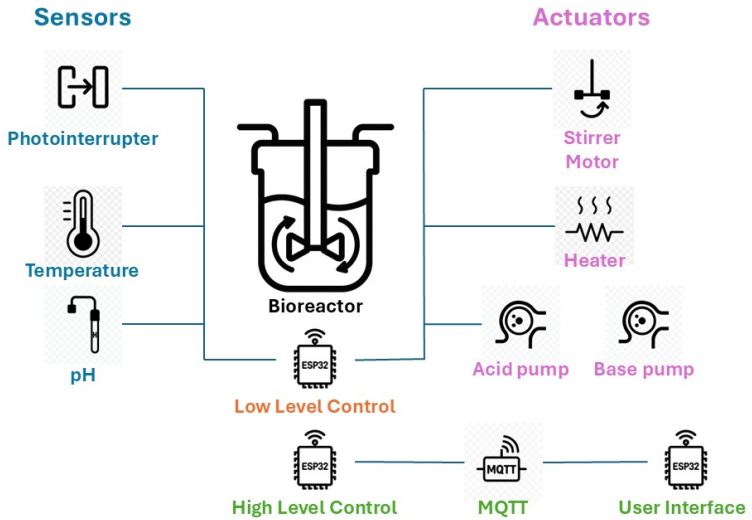
- To build the curve:
  1. Sort all samples by their anomaly score (highest first).
  2. For each possible threshold:
     - Label points above the threshold as *anomalies*.
     - Compute TP, FP, FN, then precision and recall.
  3. Plot precision (vertical axis) against recall (horizontal axis).
- The **average precision (AP)**, which approximates the area under the curve, summarises overall performance.
- A good detector maintains high precision even as recall increases.

Precision-Recall Curve

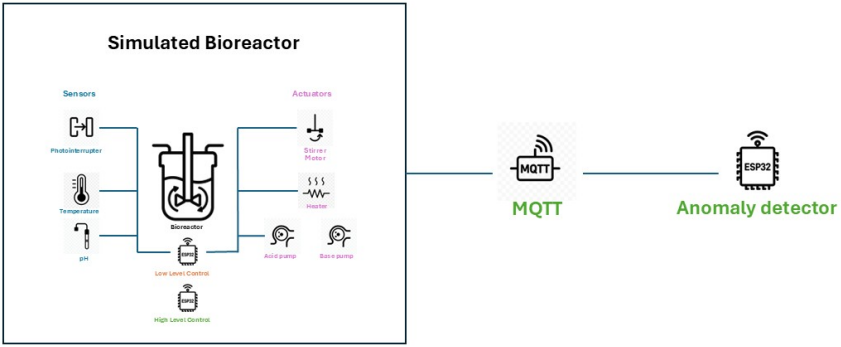


Bioreactor Simulator Context

The Bioreactor



The Simulator



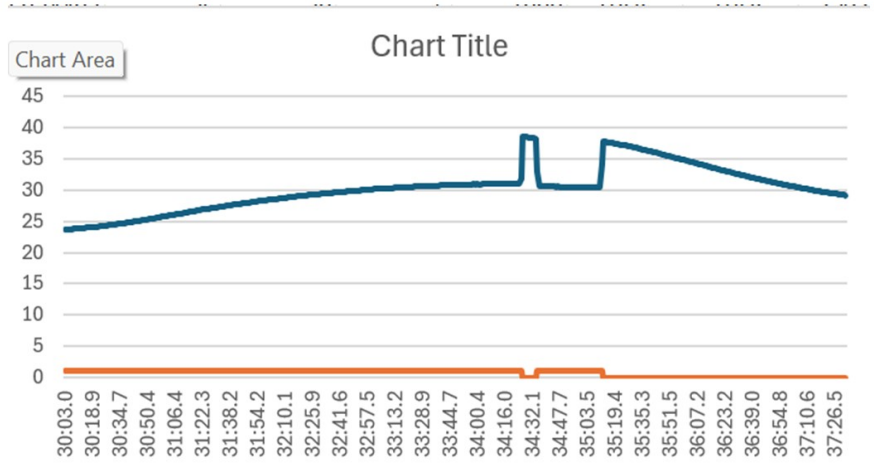
What the Simulator Does

- You do not need its code, but it behaves like a real bioreactor[1], so physical reasoning applies. It has:
  - sensors: temperature, pH, RPM;
  - actuators: heater, stirrer motor, acid and base pumps;
  - a tuned PID controller for each of temperature, pH and stirring speed.
- It publishes a **summary JSON** message over MQTT once per second.
- The temperature and pH it publishes are what the sensors **measured**, exactly what the controller saw, not the true values.
- In the simulator:
  - noise is added, and faults of different types appear and clear at random;
  - on one stream, the setpoints change on a schedule;
  - fault labels can be reported . . . or not.

[1] This is done carefully; the simulator uses principled physical and chemical models of the real world

Be careful...

It's not quite as simple as it seems.... a fault that causes the thermistor to read low.



Detector Workflow

1. **Train:** on fault-free data, fit your prediction $\hat{x}_t$, then estimate the centre m_i and spread σ_i of its residuals.
2. **Test (labelled):** compare your alarms with the fault labels, and tune τ and your alarm rule.
3. **Test (unlabelled):** deploy the detector; interpret its alarms without ground truth.

Your Team's Task

- Detect the faults![2]
- Topics: `bioreactor_sim/<STREAM>/telemetry/summary`
  - `nofaults`: train your detector;
  - `single_fault`: temperature sensor bias, **required**;
  - `three_faults`: three fault types, optional;
  - `variable_setpoints`: three fault types and changing setpoints, a stretch goal.
- Keep score second by second (TP, TN, FP, FN), then ask whether that is a fair way to judge a detector of a system under feedback control.
- Demonstrate your detector live in the final session, on a stream **without** fault labels.

[2] Full details are in the exercise handout and on Moodle.

Getting Live Data via MQTT

Understanding the MQTT Stream

Telemetry summary message (JSON):

```
{"window": {"start": 1760286536, "end": 1760286537, "seconds": 1, "samples": 11},

"temperature_C": {"mean": 22.37, "min": 22.35, "max": 22.40},

"pH": {"mean": 7.07, "min": 6.90, "max": 7.14},

"rpm": {"mean": 291.31, "min": 274.17, "max": 307.03},

"actuators_avg": {"heater_pwm": 0.99, "motor_pwm": 0.32, "acid_pwm": 0.0050, "
      base_pwm": 0.00038}, "dosing_l": {"acid": 3.35e-05, "base": 1.79e-05}, "
      heater_energy_Wh": 0.039, "photoevents": 11,

"setpoints": {"temperature_C": 35.0, "pH": 6.9, "rpm": 300.0},

"faults": {"last_active": [{"name": "therm_voltage_bias", "id": 3}], "counts": {"
      therm_voltage_bias": 11}}}
```

Key Points:

- One JSON message per second.
- `window`: time range and sample count.
- `temperature_C`, `pH`, `rpm`: measured mean, min, max.
- `actuators_avg`: PWM duty cycles.
- `faults`: labelled streams only.

ESP32 MQTT Client with PicoMQTT

- Demonstration of a minimal ESP32 MQTT client using:
  - `WiFi.h`: to connect to the wireless network;
  - `PicoMQTT.h`: a lightweight MQTT client library.
- Connects to `engf0001.cs.ucl.ac.uk`
- Subscribes to a telemetry topic and prints messages
- You should install the PicoMQTT library before compiling the code

Minimal PicoMQTT Client

```
#include <WiFi.h>
#include <PicoMQTT.h>

const char* WIFI_SSID = "YOUR_SSID";
const char* WIFI_PASSWORD = "YOUR_PASS";
const char* MQTT_HOST = "engf0001.cs.ucl.ac.uk";
const uint16_t MQTT_PORT = 1883;
const char* TOPIC = "bioreactor_sim/nofaults/telemetry/summary";

PicoMQTT::Client mqtt(MQTT_HOST, MQTT_PORT);
```

Message handler

```
void onMessage(const char* topic, const char* payload) {
  Serial.print("[MQTT] ");
  Serial.print(topic);
  Serial.print(": ");
  if (payload) {
    Serial.println(payload);
  } else {
    Serial.println("(null)");
  }
}
```

Setup and subscription

```
void setup() {
  Serial.begin(115200);

  // Connect Wi-Fi
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(250);
    Serial.print('.');
  }
  Serial.println("\nWi-Fi connected");

  // Subscribe once: PicoMQTT re-subscribes after reconnects
  mqtt.subscribe(TOPIC, onMessage);
  mqtt.begin(); // start the MQTT client
}
```

Main Loop

```
void loop() {
  mqtt.loop(); // manages reconnections and messages
}
```

Summary

- PicoMQTT makes ESP32 MQTT code very short.
- Automatic reconnects and re-subscriptions reduce complexity.
- It comfortably handles messages of a few hundred bytes, such as this JSON telemetry.
- Ideal for quick prototypes.

Wrap-Up

Key Takeaways

- A detector sees **anomalies**; we want to know about **faults**. They do not line up exactly.
- Score **residuals**, not raw readings: a detector is only as good as its prediction.
- Under feedback control, use what the controller is doing as an input to the prediction.
- Estimate centre and spread **robustly**, once, on fault-free data.
- Turn scores into stable alarms with thresholds, hysteresis and windows.
- Judge a detector with precision, recall and F_1, and think about what counts as catching a fault.

Questions?